

Unit 3: Introduction to SQL (15%)

Q1. Write SQL commands for CREATE TABLE, ALTER TABLE, DROP TABLE with examples.

Answer:

1. **CREATE TABLE** – Used to create a new table in the database.

```
CREATE TABLE Student (
    Roll_No INT PRIMARY KEY,
    Name VARCHAR(50),
    Branch VARCHAR(10),
    CGPA DECIMAL(3,2)
);
```

- Creates a table Student with 4 columns and primary key on Roll_No.

2. **ALTER TABLE** – Used to modify an existing table.

- **Add column:**

```
ALTER TABLE Student ADD Email VARCHAR(50);
```

- **Drop column:**

```
ALTER TABLE Student DROP COLUMN Email;
```

- **Modify column:**

```
ALTER TABLE Student MODIFY CGPA DECIMAL(4,2);
```

3. **DROP TABLE** – Deletes a table and all its data.

```
DROP TABLE Student;
```

In short:

- CREATE TABLE → create table
- ALTER TABLE → modify table
- DROP TABLE → delete table

Q2. Explain constraints in SQL: PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK. (IMP)

Answer:

- In SQL, **constraints** are rules applied to **columns of a table**.
- They are used to ensure **accuracy, consistency, and reliability** of data in the database.
- Without constraints, invalid or duplicate data may enter the database.

Types of Constraints:

1. **PRIMARY KEY**

- A column (or a set of columns) that **uniquely identifies each row** in a table.
- It **cannot be NULL** and must be **unique**.
- Each table can have **only one primary key**.
- Example:

```
CREATE TABLE Student (
    Roll_No INT PRIMARY KEY,
    Name VARCHAR(50)
);
```

- Here, Roll_No uniquely identifies each student.

2. FOREIGN KEY

- A column in one table that **refers to the primary key** in another table.
- Used to maintain **referential integrity** (i.e., no invalid references).
- Example:

```
CREATE TABLE Student (  
    Roll_No INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Dept_ID INT,  
    FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)  
);
```
- Here, Dept_ID in Student table must exist in Department table.

3. UNIQUE

- Ensures that all values in a column are **different** (no duplicates).
- Unlike primary key, a table can have **multiple UNIQUE constraints**.
- Example:

```
CREATE TABLE Student (  
    Email VARCHAR(50) UNIQUE  
);
```
- No two students can have the same email.

4. NOT NULL

- Ensures that a column **cannot have NULL (empty) values**.
- It forces a column to always contain valid data.
- Example:

```
CREATE TABLE Student (  
    Name VARCHAR(50) NOT NULL  
);
```

Every student must have a name.

5. CHECK

- Ensures that column values satisfy a specific **condition or rule**.
- Example:

```
CREATE TABLE Student (  
    CGPA DECIMAL(3,2) CHECK (CGPA >= 0 AND CGPA <= 10)  
);
```
- Ensures CGPA value is between 0 and 10.

In Short:

- **PRIMARY KEY** → Unique + Not Null (identifies row).
- **FOREIGN KEY** → References another table's primary key.
- **UNIQUE** → No duplicate values allowed.
- **NOT NULL** → No empty values allowed.
- **CHECK** → Values must satisfy given condition.

Importance of Constraints:

- Maintain **data integrity** (correctness of data).

- Avoid **duplicates and invalid entries**.
- Ensure **relationships** between tables are valid.
- Make database more **secure and reliable**.

Q3. Write SQL queries using aggregate functions (SUM, COUNT, AVG, MIN, MAX) with GROUP BY and HAVING.

Answer:

Aggregate Functions – Perform calculations on a set of values.

1. **SUM** – Total of a numeric column.
SELECT SUM(CGPA) AS TotalCGPA FROM Student;
2. **COUNT** – Number of rows.
SELECT COUNT(*) AS TotalStudents FROM Student;
3. **AVG** – Average value of a column.
SELECT AVG(CGPA) AS AverageCGPA FROM Student;
4. **MIN / MAX** – Minimum or maximum value.
SELECT MIN(CGPA) AS MinCGPA, MAX(CGPA) AS MaxCGPA FROM Student;

- **Using GROUP BY:**

- Example: Average CGPA per branch
SELECT Branch, AVG(CGPA) AS AvgCGPA
FROM Student
GROUP BY Branch;

- **Using HAVING:**

- Example: Branches with average CGPA > 8
SELECT Branch, AVG(CGPA) AS AvgCGPA
FROM Student
GROUP BY Branch
HAVING AVG(CGPA) > 8;

In short:

- Aggregate functions summarize data.
- GROUP BY groups data by column.
- HAVING filters groups based on condition.

Q4. Write SQL queries using built-in functions: numeric, date, and string functions.

Answer:

Built-in functions are pre-defined functions in SQL used to perform calculations or manipulate data.

1. Numeric Functions:

- Perform operations on numeric data.
- **Examples:** ROUND(), CEIL(), FLOOR(), MOD()

-- Round CGPA to 1 decimal place

SELECT Name, ROUND(CGPA, 1) AS Rounded_CGPA FROM Student;

-- Find remainder when Roll_No is divided by 10

SELECT Roll_No, MOD(Roll_No, 10) AS Remainder FROM Student;

2. Date Functions:

- Perform operations on date/time values.
- **Examples:** CURRENT_DATE, DATEDIFF(), DATEADD()
 -- Show current date
 SELECT CURRENT_DATE;

 -- Calculate age of students assuming DOB column exists
 SELECT Name, DATEDIFF(YEAR, DOB, CURRENT_DATE) AS Age FROM Student;

3. String Functions:

- Manipulate strings.
- **Examples:** UPPER(), LOWER(), SUBSTRING(), CONCAT()
 -- Convert name to uppercase
 SELECT UPPER(Name) AS UpperName FROM Student;

 -- Extract first 3 characters of Name
 SELECT SUBSTRING(Name,1,3) AS ShortName FROM Student;

 -- Concatenate first and last name
 SELECT CONCAT(First_Name,' ',Last_Name) AS FullName FROM Student;

In short:

- Numeric → Calculations on numbers
- Date → Manipulate dates
- String → Manipulate text

Q5. Write SQL queries using nested subqueries with example.**Answer:**

- ✓ **Nested subquery** = A query inside another query.
- Inner query executes first.
- Outer query uses result of inner query.

Example 1: Find students with CGPA greater than average CGPA

```
SELECT Name, CGPA
FROM Student
WHERE CGPA > (SELECT AVG(CGPA) FROM Student);
```

Example 2: Find students in the department with highest number of students

```
SELECT Name, Dept_ID
FROM Student
WHERE Dept_ID = (
    SELECT Dept_ID
    FROM Student
    GROUP BY Dept_ID
    ORDER BY COUNT(*) DESC
    LIMIT 1
);
```

In short:

- Nested subqueries are **useful for complex queries**.

- Inner query runs first, outer query uses its result.

Q6. Explain set operations in SQL: UNION, INTERSECT, EXCEPT with examples.

Answer:

Set operations combine results of **two or more queries**.

1. UNION

- Combines rows of two queries **removing duplicates**.
- **Example:** List all students and alumni names

```
SELECT Name FROM Student
UNION
SELECT Name FROM Alumni;
```

2. INTERSECT

- Returns rows **common** to both queries.
- **Example:** Students who are also alumni

```
SELECT Name FROM Student
INTERSECT
SELECT Name FROM Alumni;
```

3. EXCEPT (or MINUS in some DBMS)

- Returns rows in **first query but not in second**.
- **Example:** Students who are not alumni

```
SELECT Name FROM Student
EXCEPT
SELECT Name FROM Alumni;
```

Important points:

- Number and data type of columns must be **same** in both queries.
- Helps to **combine or filter datasets** efficiently.

In short:

- **UNION** → combine all, remove duplicates
- **INTERSECT** → only common rows
- **EXCEPT** → only first query rows not in second

Q7. Explain NULL values in SQL. How are they handled?

Answer:

- **NULL value** in SQL represents **missing, unknown, or inapplicable data** in a table.
- It is **different from 0 or empty string**.

Key points about NULL:

1. Meaning:

- Unknown or missing value.
- Example: A student may not have provided their Email.

2. Handling NULL values:

- **IS NULL / IS NOT NULL:** Used to check for NULL.

```
SELECT Name FROM Student WHERE Email IS NULL;
```
- **COALESCE():** Replaces NULL with a default value.

```
SELECT Name, COALESCE(Email, 'Not Provided') FROM Student;
```

- **IFNULL() / NVL():** DBMS-specific function to handle NULL.
SELECT Name, IFNULL(CGPA, 0) FROM Student;

3. Important points:

- NULL **cannot be compared using = or <>**. Must use IS NULL.
- Aggregate functions usually **ignore NULL** (except COUNT(*)).
- Helps in **data integrity** and proper query results.

In short:

- NULL = unknown/missing value
- Use **IS NULL**, **COALESCE()**, or similar functions to handle NULL
- Essential for proper **data handling in SQL**

Q8. Explain DCL (GRANT, REVOKE) and TCL (COMMIT, ROLLBACK, SAVEPOINT) with examples. (IMP)

Answer:

1. DCL – Data Control Language

- Used to **control access and permissions** in a database.

Commands:

1. GRANT – Gives privileges to users.

-- Give *SELECT* and *INSERT* permission on Student table to user 'user1'
GRANT SELECT, INSERT ON Student TO user1;

2. REVOKE – Removes privileges from users.

-- Remove *INSERT* permission from user1
REVOKE INSERT ON Student FROM user1;

Purpose:

- Ensures **security** and proper access control in a multi-user environment.

2. TCL – Transaction Control Language

- Used to **manage transactions** in the database.
- A **transaction** = a group of SQL statements executed as a single unit.

Commands:

1. COMMIT – Saves all changes permanently.

INSERT INTO Student VALUES (103, 'Priya', 'CSE', 3.7);
COMMIT;

2. ROLLBACK – Undo changes made in the current transaction.

UPDATE Student SET CGPA = 3.0 WHERE Roll_No = 102;
ROLLBACK; -- Undo the update

3. SAVEPOINT – Set a point within a transaction to roll back partially.

SAVEPOINT sp1;
UPDATE Student SET CGPA = 3.5 WHERE Roll_No = 101;
ROLLBACK TO sp1; -- Only undo changes after sp1

Key points:

- TCL ensures **data consistency and integrity**.
- DCL ensures **security and controlled access**.

In short:

- **DCL** → Control who can access/modify the database (GRANT, REVOKE)
- **TCL** → Control transactions to **commit, undo, or partially rollback** changes

Q9. Explain nested subqueries in SQL with examples

Answer:

- A **nested subquery** (or inner query) is a **query inside another query** (outer query).
- The **inner query executes first**, and its result is used by the outer query.
- Nested subqueries are useful for **complex queries** where filtering depends on another query.

Syntax Example:

```
SELECT column1
FROM table1
WHERE column2 = (SELECT column2 FROM table2 WHERE
condition);
```

Examples:

1. Find students with CGPA greater than average CGPA

```
SELECT Name, CGPA
FROM Student
WHERE CGPA > (SELECT AVG(CGPA) FROM Student);
```

- Inner query: (SELECT AVG(CGPA) FROM Student) → calculates average CGPA.
- Outer query: Selects students **above average CGPA**

2. Find students in the department with maximum students

```
SELECT Name, Dept_ID
FROM Student
WHERE Dept_ID = (
    SELECT Dept_ID
    FROM Student
    GROUP BY Dept_ID
    ORDER BY COUNT(*) DESC
    LIMIT 1
);
```

- Inner query finds **Dept_ID with most students**.
- Outer query lists **students in that department**.

Key Points:

- Inner query executes **first**, outer query uses its result.
- Can be used in **WHERE, FROM, or SELECT clauses**.
- Helps solve **complex conditional queries** efficiently.

In short: Nested subquery = **query inside a query**, used for **dependent filtering**.

Q10. Write SQL queries using aggregate functions with GROUP BY and HAVING. (IMP)

Answer:

- **Aggregate functions** perform **calculations on a set of rows** and return a single value.
- Common functions: SUM(), COUNT(), AVG(), MIN(), MAX().
- **GROUP BY:** Groups rows with the **same value** in specified columns.
- **HAVING:** Filters groups after aggregation (like WHERE for grouped data).

Example 1: Total students in each department

```
SELECT Dept_ID, COUNT(*) AS Total_Students  
FROM Student  
GROUP BY Dept_ID;
```

- GROUP BY Dept_ID → Groups students by department.
- COUNT(*) → Counts number of students in each department.

Example 2: Departments with average CGPA greater than 8

```
SELECT Dept_ID, AVG(CGPA) AS Avg_CGPA  
FROM Student  
GROUP BY Dept_ID  
HAVING AVG(CGPA) > 8;
```

- HAVING AVG(CGPA) > 8 → Filters only departments with **average CGPA > 8**.

Key Points:

- **Aggregate functions** summarize data efficiently.
- **GROUP BY** organizes data into groups.
- **HAVING** filters **grouped results** after aggregation.
- Cannot use WHERE to filter aggregated results; use HAVING.

In short:

- GROUP BY = Group rows by column
 - HAVING = Filter after aggregation
 - Aggregate functions = SUM, COUNT, AVG, MIN, MAX
-

Unit 4: Entity-Relationship Model (10%)

Q1. Define entity, entity set, relationship, relationship set, attributes with examples

Answer:

1. Entity:

- An entity is a **real-world object** that is **distinct and identifiable**.
- Can be **tangible** (like a student) or **intangible** (like a course).
- **Example:** A student Rahul is an entity.

2. Entity Set:

- A **collection of similar entities** sharing the same attributes.
- Represented as a **table** in relational model.
- **Example:** Student = {Rahul, Amit, Priya}

3. Attribute:

- A property or characteristic of an entity.
- Types of attributes:
 - **Simple:** Cannot be divided further (e.g., Name)
 - **Composite:** Can be divided into sub-parts (e.g., FullName → FirstName, LastName)
 - **Derived:** Can be derived from other attributes (e.g., Age from DOB)
 - **Multivalued:** Can have multiple values (e.g., PhoneNumbers)
- **Example:** Roll_No, Name, CGPA of a student

4. Relationship:

- A relationship represents **association between entities**.
- **Example:** Student Rahul **enrolled in** Course DBMS.

5. Relationship Set:

- A **collection of similar relationships** of the same type.
- **Example:** All enrollments of students in courses:
- { (Rahul, DBMS), (Amit, Maths), (Priya, DBMS) }

Diagram Example:

Student(Roll_No, Name) ---- Enrolled_in ---- Course(Course_ID, Name)

In short:

- Entity → Real-world object
- Entity Set → Collection of similar entities
- Attribute → Property of entity
- Relationship → Association between entities
- Relationship Set → Collection of similar relationships

Q2. Explain relationship constraints: cardinality and participation

Answer:

Relationship constraints define rules about how entities participate in a relationship.

1. Cardinality Constraint:

- Specifies the **number of entities** that can participate in a relationship.
- Types:
 - **One-to-One (1:1):** Each entity in A is related to **only one** entity in B and vice versa.
 - Example: A student has one locker.

- **One-to-Many (1:N):** Each entity in A is related to **many entities** in B, but B entity relates to only one A entity.
 - Example: A **department has many students**, each student belongs to one department.
- **Many-to-Many (M:N):** Entities in A can relate to **many entities in B** and vice versa.
 - Example: **Students enroll in multiple courses** and **courses have multiple students**.

2. Participation Constraint:

- Specifies whether **all or some entities must participate** in a relationship.
- Types:
 - **Total Participation:** Every entity must participate (depicted by double line).
 - Example: Every student must be enrolled in at least one course.
 - **Partial Participation:** Some entities may not participate (depicted by single line).
 - Example: Some students may not be assigned a mentor.

Diagram Example:

Student ---- Enrolled_in ---- Course
 |1 M| M:N
 (Total participation)

Key Points:

- Cardinality → **How many** entities can participate
- Participation → **Whether all or some** entities must participate
- Helps maintain **integrity of relationships** in database design

Q3. What are weak entities? Explain with an example

Answer:

- A **weak entity** is an entity that **cannot be uniquely identified by its own attributes alone**.
- It **depends on a strong entity** (owner) for its identification.
- Weak entities have a **partial key**, which combined with the primary key of the owner entity forms a **composite key**.
- **Relationship connecting weak entity to owner** is called an **identifying relationship**.

✓ Characteristics of Weak Entity:

1. No **primary key** of its own.
2. **Exists only if the owner entity exists**.
3. Represented as a **double rectangle** in ER diagram.
4. Identifying relationship is represented as a **double diamond**.

Example:

- **Entities:**
 - Department (strong entity, PK: Dept_ID)
 - EmployeeDependent (weak entity, Partial key: Dep_Name)
- Each dependent **belongs to exactly one employee** in a department.
- Composite key: (Dept_ID + Dep_Name) uniquely identifies a dependent.

ER Diagram Example:

[Department]---◆---[EmployeeDependent]
 Dept_ID(PK) Dep_Name(Partial Key)

Explanation:

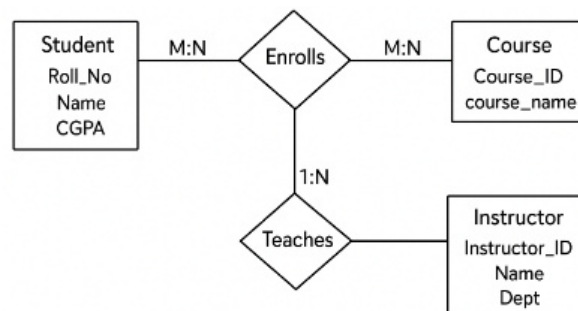
- EmployeeDependent cannot be uniquely identified without Dept_ID.
- Weak entity depends on the **owner entity** Department.

In short:

- Weak entity = **depends on strong entity**
- Has **partial key** + identifying relationship
- Example: Employee dependents, insurance claims, orders in a customer database

Q4. Draw an E-R diagram for a Student–Course–Instructor scenario (IMP)**Scenario:**

- **Entities:** Student, Course, Instructor
- **Relationships:**
 1. Student **enrolls in** Course (M:N)
 2. Instructor **teaches** Course (1:N)
- **Attributes:**
 - Student: Roll_No (PK), Name, CGPA
 - Course: Course_ID (PK), Course_Name
 - Instructor: Instructor_ID (PK), Name, Dept

ER Diagram:**Explanation:**

1. **Student–Course (M:N):**
 - A student can enroll in many courses.
 - A course can have many students.
 - Requires a **junction table** when converted to relational schema.
2. **Instructor–Course (1:N):**
 - An instructor can teach many courses.
 - A course is taught by **one instructor**.

Additional Notes:

- Attributes are represented inside rectangles.
- Relationships are diamonds connecting entities.
- Cardinality is shown near lines (1:N, M:N).
- Helps in **database design and normalization**.

In short:

- E-R diagram visually shows **entities, attributes, relationships, and cardinalities**.
- Important for designing relational database schemas.

Q5. How is an E-R schema reduced to relational schema? Explain steps**Answer:**

- An **E-R schema** is a conceptual design of a database.
- **Relational schema** is the logical design represented as tables for implementation.
- **Reducing ER schema to relational schema** is necessary to implement the database in RDBMS.
- **Steps to Reduce ER Schema:**
 1. **Convert Strong Entities to Tables:**
 - Each **strong entity** becomes a table.
 - Attributes of the entity become **columns**.
 - Primary key of entity becomes **primary key** of table.
 - **Example:**
Entity: Student(Roll_No, Name, CGPA)
Table: Student(Roll_No PK, Name, CGPA)
 2. **Convert Weak Entities to Tables:**
 - Weak entity becomes a table.
 - Include **primary key of owner entity** as foreign key.
 - Composite key = partial key + owner's primary key.
 - **Example:**
Entity: EmployeeDependent(Dep_Name, Employee_ID)
Table: EmployeeDependent(Employee_ID FK, Dep_Name, PRIMARY KEY(Employee_ID, Dep_Name))
 3. **Convert Relationships to Tables:**
 - **1:1 relationship:** Can merge into one table or create a separate table with foreign keys.
 - **1:N relationship:** Include **primary key of "1" side as foreign key** in "N" side table.
 - **M:N relationship:** Create a new table with **primary keys of both entities as foreign keys**, forming **composite primary key**.
 - **Example:**
Student–Course (M:N) → Enrollment(Roll_No FK, Course_ID FK, PRIMARY KEY(Roll_No, Course_ID))
 4. **Include Attributes of Relationships:**
 - If relationship has attributes, include them in **table for that relationship**.
 - Example: Enrollment(Date, Grade) → Table: Enrollment(Roll_No FK, Course_ID FK, Date, Grade)

In short:

- **Strong entity** → **table**
- **Weak entity** → **table + owner key**
- **Relationship** → **table or foreign key**
- Attributes included in respective tables

Q6. Explain how to remove redundant attributes in entity sets**Answer:**

- Redundant attributes are attributes that do not provide new information or can be derived from other attributes.
 - Removing them improves **database efficiency** and reduces **data redundancy**.
- ✓ **Steps to Remove Redundant Attributes:**
1. **Identify Derived Attributes:**
 - Attributes that can be calculated from others.
 - **Example:** Age can be derived from DOB → remove Age column.
 2. **Check for Repeating or Duplicate Attributes:**
 - Attributes stored in multiple tables unnecessarily.
 - Keep only one copy in the **appropriate table**.
 3. **Remove Transitive Redundancy:**
 - Attributes that depend on **non-key attributes** instead of primary key.
 - Example: In Student table, storing Dept_Name along with Dept_ID is redundant → Dept_Name can be obtained from Department table.
 4. **Update the ER Diagram or Schema:**
 - After removing redundant attributes, **update the entity or relationship diagram**.

In short:

- Remove attributes that are **derivable, repeating, or dependent on non-key attributes**
- Reduces storage and ensures **data consistency**

Q7. Explain steps to reduce an ER schema to relational schema (IMP)**Answer:**

This is a **stepwise approach combining Q5 & Q6**, exam-friendly.

Step 1: Strong Entities → Tables

- Create a table for each strong entity.
- Attributes become columns.
- Primary key of entity → primary key of table.

Step 2: Weak Entities → Tables

- Include primary key of owner entity as foreign key.
- Combine with partial key to form **composite primary key**.

Step 3: Relationships → Tables / Foreign Keys

- **1:1 relationship:** Include foreign key in one of the tables or separate table.
- **1:N relationship:** Foreign key in “N” side table.
- **M:N relationship:** Create new table with primary keys of both entities as composite primary key.

Step 4: Include Relationship Attributes

- Attributes of relationships included in **corresponding tables**.

Step 5: Remove Redundant Attributes

- Remove **derived, duplicate, or transitive attributes**.
- Update schema to avoid **redundancy**.

Step 6: Final Relational Schema

- Each table should have:
 - Primary key

- Foreign keys (if needed)
- Relevant attributes
- Ensures **referential integrity** and **efficient storage**

Example: Student–Course–Instructor scenario

Table	Columns	Key
Student	Roll_No, Name, CGPA	PK: Roll_No
Course	Course_ID, Course_Name	PK: Course_ID
Instructor	Instructor_ID, Name, Dept	PK: Instructor_ID
Enrollment	Roll_No, Course_ID, Grade	PK: Roll_No+Course_ID, FK: Roll_No→Student, Course_ID→Course
Teaches	Course_ID, Instructor_ID	PK: Course_ID, FK: Instructor_ID→Instructor